

Système d'exploitation

Système d'exploitation - Utilisation

Mickael Rouvier

CERI – Avignon Université
`mickael.rouvier@univ-avignon.fr`

February 10, 2025

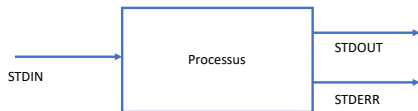
Section 1

Redirection de flux

Les entrées/sorties des processus

Chaque processus possède 3 flux standards qu'il utilise pour communiquer en général avec l'utilisateur :

- l'**entrée standard** nommée : *stdin* (identifiant du flux : 0) – il s'agit par défaut du clavier
- la **sortie standard** nommée : *stdout* (identifiant du flux : 1) – il s'agit par défaut de l'écran
- la **sortie d'erreur standard** nommée : *stderr* (identifiant du flux : 2) – il s'agit par défaut de l'écran



Sortie standard : stdout

Exemple : programme qui affiche sur la sortie standard (stdout)

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(int argc, char** argv) {
6
7     cout<<"Exemple de sortie standard (stdout)"<<endl;
8
9     return 0;
10 }
```

Listing 1: "Sortie standard (stdout)"

Sortie d'erreur : stderr

Exemple : programme qui affiche sur la sortie d'erreur (stderr)

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(int argc, char** argv) {
6
7     cerr<<"Exemple de sortie d'erreur (stderr)"<<endl;
8
9     return 0;
10 }
```

Listing 2: "Sortie d'erreur (stderr)"

Entrée standard : stdin

Exemple : programme qui récupère les informations sur l'entrée standard (stdin)

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(int argc, char** argv) {
6
7     string x;
8     cin>>x;
9
10    return 0;
11 }
```

Listing 3: "Entrée standard (stdin)"

Rediriger la sortie standard

- **Redirection :**

- Quand on exécute une commande, le shell affiche le résultat sur la console de sortie (l'écran par défaut).
- On peut rediriger cette sortie vers un fichier en utilisant le signe >

```
invite/> ./programme > file.txt
```

- **Concaténation :**

- Au lieu de créer un fichier, il est possible d'ajouter les sorties d'un processus à un fichier existant en utilisant le double signe »

```
invite/> ./programme >> file.txt
```

- **Syntaxe complète :**

- Les signes > peuvent être précédés de l'identifiant du flux à rediriger. Pour la sortie standard, on peut donc utiliser les syntaxes suivantes.

```
invite/> ./programme 1> file.txt
```

```
invite/> ./programme 1>> file.txt
```

Rediriger la sortie d'erreur standard

- **Redirection :**

- La redirection du flux de sortie d'erreur standard utilise les même signes, mais précédés de l'identifiant du flux : 2.

```
invite/> ./programme 2> ls_erreur.txt  
invite/> ./programme 2>> ls_erreur.txt
```


Rediriger l'entrée standard

- **Redirection :**

- Rediriger l'entrée standard permet d'entrer des données provenant d'un fichier au lieu du clavier.

```
invite/> cat < mon_fichier.txt
```

- **Redirection :**

- Il est possible de rediriger un flux vers la sortie standard ou la sortie d'erreur en donnant l'identifiant du flux précédé du caractère & à la place du nom de fichier.

```
invite/> ./programme 1>stdout_stderr.txt 2>&1
```

- Le fichier stdout_stderr.txt contient ce qui a été affiché à la fois sur le flux de sortie standard et le flux de sortie d'erreur.

Exemple : calculer la moyenne – Partie 1

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main(int argc, char** argv) {
7      bool loop = true;
8      float mean = 0.0;
9      float number = 0.0;
10
11     cout<<"Entrez vos notes : " <<endl;
12     while(loop) {
13         string xx;
14         cin >> xx;
15
16         if (xx != "exit") {
17             if (stof(xx) < 0.0) {
18                 cerr<<"Note inattendu !" <<endl;
19             }
20             else {
21                 mean += stof(xx);
22                 number += 1.0;
23             }
24         }
25         else {
26             loop = false;
27         }
28     }
29
30     cout<<"Votre moyenn est de : " <<mean/number<<endl;
31     return 0;
32 }
```

Exemple : calculer la moyenne – Partie 2

```
invite/> ./moyenne
Entrez vos notes :
11
12
-5
Note inattendu !!
13
-1
Votre moyenne : 12
```

Exemple : calculer la moyenne – Partie 3

```
invite/> cat note.txt
```

```
11  
12  
-5  
13  
-1
```

```
invite/> ./moyenne < note.txt
```

```
Entez vos notes :
```

```
11  
12  
-5  
Note inattendu !!  
13  
-1
```

```
Votre moyenne : 12
```

Exemple : calculer la moyenne – Partie 4

```
invite/> ./moyenne < note.txt > stdout.txt  
Note inattendu !!
```

```
invite/> cat stdout.txt  
Entez vos notes :  
11  
12  
-5  
13  
-1  
Votre moyenne : 12
```

Exemple : calculer la moyenne – Partie 5

```
invite/> ./moyenne < note.txt > stdout.txt 2> stderr.txt
```

```
invite/> cat stderr.txt
```

```
Note inattendu !!
```

Pipe : chaîne les commandes - Partie 1

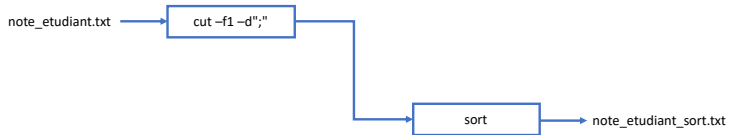
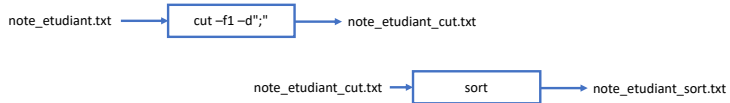
Problématique : Comment afficher uniquement les prénoms trié par ordre alphabétique ?

```
invite/> cat note_etudiant.txt  
Emma;18  
Alice;12  
Jade;5  
Léna;19  
Camille;18
```

Solution 1 : Consiste à utiliser des fichiers temporaires

```
invite/> cut -f1 -d";" note_etudiant.txt > note_etudiant_cut.txt  
invite/> sort note_etudiant_cut.txt  
Alice  
Camille  
Emma  
Jade  
Léna  
invite/> rm note_etudiant_cut.txt
```


Pipe : chaîne les commandes - Partie 1



Pipe : chaîne les commandes - Partie 2

Solution 2 : Utiliser les pipe

```
invite/> cut -f1 -d";" note_etudiant.txt | sort  
Alice  
Camilie  
Emma  
Jade  
Léna
```

Le pipe est un mécanisme qui permet de rediriger la sortie (stdout) d'un programme vers l'entrée (stdin) d'un autre programme

```
invite/> cat note_etudiant.txt | cut -f1 -d";" | sort  
Alice  
Camilie  
Emma  
Jade  
Léna
```

Pipe : chaîne les commandes - Partie 3

Soit une matrice carré d'ordre 4 :

```
invite/> cat matrix.txt  
01;02;03;04  
04;05;06;07  
08;09;10;11  
12;13;14;15
```

Afficher la première ligne du fichier

```
invite/> cat matrix.txt | head -n 1
```

Afficher la dernière ligne du fichier

```
invite/> cat matrix.txt | tail -n 1
```

Afficher la troisième ligne du fichier

```
invite/> cat matrix.txt | head -n 3 | tail -n 1
```

Afficher l'élément $A_{3,3}$

```
invite/> cat matrix.txt | head -n 3 | tail -n 1 | cut -f3 -d";"
```

Pipe : exemples d'utilisation

Des exemples d'utilisation de pipe avec la commande history :

```
invite/> history | tail -n 3
invite/> history | grep red
invite/> history | grep red | grep blue
invite/> history | grep red | more
invite/> history | more
```

Des exemples d'utilisation de pipe avec la commande ls :

```
invite/> ls -lrth | tail -n 3
invite/> ls -lrth | head -n 5
invite/> ls -lrth | more
```

Des exemples d'utilisation de pipe avec des fichiers :

```
invite/> cat fichier.txt | tail -n 2 | head -n1
invite/> cat fichier.txt | cut -f4 -d";" | head -n 3 | tail -n 1
```

La commande **tr** permet de modifier le contenu de l'entrée standard :
Soit une matrice carré d'ordre 4 :

```
invite/> cat matrix.txt  
01;02;03;04  
04;05;06;07  
08;09;10;11  
12;13;14;15
```

Remplacer les ; par des - :

```
invite/> cat matrix.txt | tr ";" "-"  
01-02-03-04  
04-05-06-07  
08-09-10-11  
12-13-14-15
```

La commande **tr** permet de modifier le contenu de l'entrée standard :
Soit une matrice carré d'ordre 4 :

```
invite/> cat fichier.txt  
toto  
tata  
titi  
tutu
```

Remplacer les ; par des - :

```
invite/> cat fichier.txt | tr [:lower:] [:upper:]  
TOTO  
TATA  
TITI  
TUTU
```

La commande **join** permet de fusionner les lignes de deux fichiers ayant un champ commun :

```
invite/> cat janvier2012  
Alimentation 50.00  
Eau 25.00  
Electricite 123.50  
Loyer 456.90  
Assurances 234.00
```

```
invite/> cat fevrier2012  
Alimentation 67.00  
Eau 34.00  
Electricite 156.00  
Loyer 456.90  
Assurances 225.00
```

La commande **join** permet de fusionner les lignes de deux fichiers ayant un champ commun :

```
invite/> join -1 1 -2 1 -t" " janvier2012 fevrier2012
Alimentation 50.00 67.00
Eau 25.00 34.00
Electricite 123.50 156.00
Loyer 456.90 456.90
Assurances 234.00 225.00
```


La commande **join** permet de fusionner les lignes de deux fichiers ayant un champ commun :

```
invite/> cat fevrier2012 | join -1 1 -2 1 -t" " janvier2012 -  
Alimentation 50.00 67.00  
Eau 25.00 34.00  
Electricite 123.50 156.00  
Loyer 456.90 456.90  
Assurances 234.00 225.00
```

Section 2

Programmation en bash

- Jusqu'à présent on a vu comment lancer des commandes
- **Problème :**
 - Comment **relancer** une séquence de commande ?
 - Comment rendre des séquences de commandes **générique** ?
- Langage de programmation intégré au bash (variable, boucle, test conditionnelle...)
- Avantage
 - Permet d'automatiser des tâches (sauvegarder vos données, surveillance de votre réseau...)
 - Rien à installer
 - Très peu de nouvelles choses à apprendre. On peut utiliser les commandes du bash simplement (*ls, cut, grep...*)
- Inconvénient
 - Pas aussi complet que le langage de programmation C/C++ ou autre (exception, pointeur...)

Création d'un script bash - Hello World

Exemple de création d'un script bash :

- Fichier texte (l'extension du fichier est .sh)
- Hashbang représenté par **#!** : en-tête du fichier qui permet de définir l'interpréteur à utiliser

```
1 #!/bin/python
2
3 print "Hello world !"
```

Listing 5: "Exemple de script en python : main.py"

```
1 #!/bin/bash
2
3 echo "Hello world !"
```

Listing 6: "Exemple de script bash : main.sh"

- Plusieurs façon d'exécuter un script :

```
invite/> chmod u+x main.py invite/> ./main.py
```

```
invite/> python main.py
```

```
invite/> chmod u+x main.sh invite/> ./main.sh
```

```
invite/> sh main.sh
```

Variables : affectation - Partie 1

Affectation basique de variable

- Typage dynamique (pas nécessaire de spécifier le type)
- **Attention** : pas d'espace avant et après le =
- La variable peut contenir des chiffres, des lettres et des symboles souligné mais ne doit pas commencer par un chiffre
- La portée de la variable est globale

```
1 int integer = 5;
2 float flt = 4.5;
3 string str = "bonjour";
```

Listing 7: "exemple de typage de variable en C++"

```
1 integer = 5
2 flt = 4.5
3 str = "bonjour"
```

Listing 8: "exemple de typage de variable en python"

```
integer=5
flt=4.5
str="bonjour"
```

Listing 9: "exemple de typage de variable en bash"

Variables : substitution – Partie 1

Afficher le contenu d'une variable :

- On peut afficher le contenu d'une variable à l'aide de la commande **echo**
- **\$** permet d'accéder au contenu de la variable
- La substitution est le mécanisme qui différencie la variable de son contenu

```
1 int a = 5;  
2 cout<< a <<endl;
```

Listing 10: "Exemple pour afficher le contenu d'une variable en C++"

```
1 a = 5  
2 print a
```

Listing 11: "Exemple pour afficher le contenu d'une variable en python"

```
1 a=5  
2 echo $a
```

Listing 12: "Exemple pour afficher le contenu d'une variable en bash"

Variables : substitution - Partie 2

Afficher le contenu d'une variable :

- **simple quote ou apostrophe** délimitent une chaîne de caractères. **Attention** : les variables ne seront pas interprétées
- **double quotes ou guillemets** délimitent une chaîne de caractère, les variables sont interprétés par le shell. Utile pour générer des messages dynamiques au sein d'un script.
- **\${}** permet de délimiter le début et la fin d'une variable

```
1 var=paul
2 echo 'bonjour $var'
```

```
bonjour $var
```

```
1 var=paul
2 echo "bonjour $var"
```

```
bonjour paul
```

```
1 var=fichier
2 echo "$var_conf.txt"
3 echo "${var}_conf.txt"
```

Variables : affectation - Partie 2

Affectation de variable élaborée :

- Affectation de variable à partir d'une autre variable

```
1 toto=bonjour
2 variable=$toto
3 toto=salut
4 echo $variable
5 echo $toto
```

```
bonjour
salut
```

```
1 extension=.txt
2 variable=fichier$toto
3 echo $variable
```

```
fichier.txt
```


Variables : affectation - Partie 3

Affectation de variable élaborée :

- Affectation de variable élaborée : affecte le résultat d'une commande

```
1 file=/etc/apache2/apache.conf
2 variable=`basename $file .conf`
3 echo $variable
```

apache

```
1 variable=`ls *.txt`
2 echo $variable
```

file1.txt fileN.txt

```
1 variable=`cat /etc/passwd | grep root | cut -f5 -d" "`
2 echo $variable
```

password

- Les back-quotes, apostrophes inversées ou accents graves permettent délimitent une commande à exécuter.

Tester une condition

Permet de tester [condition] :

renvoient le code retour 0 si l'expression est vrai

```
1 [ 2 = 2 ]  
2 echo $?
```

0

renvoient le code retour 1 si l'expression est fausse

```
1 [ 2 = 1 ]  
2 echo $?
```

1

Test - chaîne de caractère

Les opérateurs de tests disponibles pour les chaînes de caractères sont :

- **c1 = c2** : vrai si c1 et c2 sont égaux
- **c1 != c2** : vrai si c1 et c2 sont différents
- **-z c** : vrai si c est une chaîne vide
- **-n c** : vrai si c n'est pas une chaîne vide

```
1 variable=john
2 [ "john" = ${variable} ]
3 echo $?
```

0

Les opérateurs de tests disponibles pour les nombres sont :

- **n1 -eq n2** : vrai si n1 et n2 sont égaux (equal)
- **n1 -ne n2** : vrai si n1 et n2 sont différents (not equal)
- **n1 -lt n2** : vrai si n1 est strictement inférieur à n2 (less than)
- **n1 -le n2** : vrai si n1 est inférieur ou égal à n2 (less equal)
- **n1 -gt n2** : vrai si n1 est strictement supérieur à n2 (greater than)
- **n1 -ge n2** : vrai si n1 est supérieur ou égal à n2 (greater equal)

```
1 variable=5
2 [ 5 -eq ${variable} ]
3 echo $?
```

0

Les opérateurs de tests disponibles pour les objets du système sont :

- **-e \$FILE** : vrai si l'objet désigné par \$FILE existe
- **-s \$FILE** : vrai si l'objet désigné par \$FILE existe et sa taille est supérieure à zéro
- **-f \$FILE** : vrai si l'objet désigné par \$FILE est un fichier dans le répertoire courant

```
1 [ -e "fichier.conf" ]  
2 echo $?
```

```
0
```

- Les opérateurs de tests disponibles sont, pour les expressions régulières :
 - **! e** : vrai si e est faux
 - **e1 -a e2** : vrai si e1 et e2 sont vrais
 - **e1 -o e2** : vrai si e1 ou e2 est vrai

La structure conditionnelle if - Partie 1

- L'instruction **if** permet d'effectuer les instructions si la condition est réalisée (VRAI) :

```
1 if condition
2 then
3     instructions
4 fi
```

- Exemple :

```
1 a=5
2 if [ $a -eq "5" ]
3 then
4     echo "a is equal to 5"
5 fi
```

```
a is equal to 5
```

La structure conditionnelle if - Partie 2

L'instruction **if** peut aussi inclure une instruction **else** permettant d'exécuter des instructions dans le cas où la condition n'est pas réalisée (FAUX) :

```
1  if condition
2  then
3      instructions
4  else
5      instructions
6  fi
```


La structure conditionnelle if - Partie 3

Exemple :

```
1 a=7
2 if [ $a -eq "5" ]
3 then
4     echo "a is equal to 5"
5 else
6     echo "a is not equal to 5"
7 fi
```

```
a is not equal to 5
```

La structure conditionnelle if - Partie 4

Il est bien sur possible d'imbriquer des if dans d'autres if :

```
1  if condition
2  then
3      instructions
4  else
5      if condition
6      then
7          instructions
8      else
9          instructions
10     fi
11 fi
```

La structure conditionnelle case

- L'instruction **case** permet de comparer une valeur avec une liste d'autres valeurs et d'exécuter un bloc d'instructions lorsqu'une des valeurs de la liste correspond.

```
1 case valeur_testee in
2     valeur1) instruction(s);;
3     valeur2) instruction(s);;
4     valeur3) instruction(s);;
5 esac
```

- Les valeurs peuvent contenir des caractères joker, ainsi :

```
1 case $a in
2     t?t?) echo "ok";;
3     *) exit 1;;
4 esac
```

- affichera ok si la variable \$a a une valeur correspondant au motif t?t?, comme par exemple toto ou tata (tonton ne marchera pas).

Boucle for – Partie 1

- La boucle for permet de parcourir une liste de valeurs (elle effectue donc un nombre de tours de boucle qui est connu à l'avance)

```
1 for variable in liste_valeurs
2 do
3     instructions
4 done
```

- Exemple en mettant des valeurs :

```
1 for valeur in e1 e2 e3
2 do
3     echo $valeur
4 done
```

Boucle for – Partie 2

- Exemple en utilisant la commande seq :

```
1 for valeur in `seq 1 1 5`  
2 do  
3     echo "boucle $valeur"  
4 done
```

- Exemple en utilisant la commande ls :

```
1 for valeur in `ls a*`  
2 do  
3     echo "fichier $valeur"  
4 done
```

Boucle for – Partie 3

- Exemple en utilisant les arguments de script :

```
1 for valeur in $@  
2 do  
3     echo "argument $valeur"  
4 done
```

Boucle for – Partie 4

- Dans laquelle e1, e2 et e3 sont des expressions arithmétiques. Une telle boucle commence par exécuter l'expression e1, puis tant que l'expression e2 est différente de zéro le bloc de l'instruction est exécutée et l'expression e3 évaluée

```
1 for ((e1; e2; e3))  
2 do  
3     instruction  
4 done
```

- Exemple deuxième syntaxe

```
1 for ((i=0; 10 - $i, i++))  
2 do  
3     echo "iteration $i"  
4 done
```

Boucle while - Partie 1

- La boucle while exécute un bloc d'instructions tant qu'une certaine condition est satisfaisante, lorsque cette condition devient fausse la boucle se termine. Cette boucle permet donc de faire un nombre indéterminé de tours de boucle, voir infini si la condition ne devient jamais fausse.
- Exemple :

```
1 while condition
2 do
3     instruction
4 done
```


Boucle while - Partie 2

- Exemple d'utilisation :

```
1  atrouver=50
2  echo "entrez un nombre compris entre 1 et 100"
3  read i
4  while [ "$i" -ne "$atrouver" ]
5      if [ "$i" -lt "$atrouver" ]
6      then
7          echo "trop petit, entrez un nombre compris entre 1 et 100"
8      else
9          echo "trop grand, entrez un nombre compris entre 1 et 100"
10     fi
11     read i
12 done
```

Récupérer les arguments passés dans un script - Partie 1

- Plusieurs variables spéciales sont disponibles lors de l'exécution d'un script.
 - **\$0** a pour valeur le nom du script
 - **\$1** jusqu'à **\$9** ont respectivement pour valeur les neuf premiers
 - **\$#** a pour valeur le nombre d'arguments passés au script
 - **\$@** contient la liste de tous les arguments du script
- Exemple pour afficher les arguments

```
1 echo $1
2 echo $2
```

```
invite/> ./programme.sh toto tata
toto
tata
```

Récupérer les arguments passés dans un script - Partie 2

- Exemple pour vérifier le nombre d'argument

```
1  if [ "$#" != "2" ]
2  then
3      echo "$0 <(i) param_1> <(i) param_2>"
4      echo "param_1 : parametre 1"
5      echo "param_2 : parametre 2"
6      exit -1
7  fi
8
9  echo $1
10 echo $2
```

Problématique : Comment effectuer des calculs via le bash ?

- La syntaxe **`$((operation))`**

```
1 a=1
2 a=$((a+1))
3 echo $a
```

- La commande **let**

```
1 a=1
2 let "a=$a + 1"
3 echo $a
```

- La commande **bc**

```
1 a=1
2 a=`echo "$a+1" | bc`
3 echo $a
```

- En natif, bash ne propose que des fonctionnalités de calcul limitées (additions simples...). bc permet des calculs plus complexes, avec gestion des décimales (ne pas oublier l'option -l).

```
1 echo "1/3" | bc -l
2 echo "sqrt(2)" | bc -l
```

Fonction - Déclaration de fonction

- Comme pour toute autre langage, l'utilisation de fonctions facilite le développement de scripts et la structuration de ceux-ci.
 - **Déclaration** – pour déclarer une fonction, on utilise la syntaxe suivante :

```
1 MaFunction() {  
2     instructions  
3 }
```

- **Appel et paramètres** – pour appeler une fonction, on utilise la syntaxe suivante :

```
1 MaFunction "param_1" "param_2" ... "param_N"
```

```
1 MaFunction() {  
2     param1=$1  
3     param2=$2  
4 }
```

Fonction - Exemples

- Exemple - déclarer et appeler une fonction sans argument

```
1 MaFunction() {  
2     echo "on entre dans ma fonction"  
3 }  
4  
5 MaFunction
```

- Exemple - déclarer et appeler une fonction avec argument

```
1 MaFunction() {  
2     param1=$1  
3     param2=$2  
4 }  
5  
6 MaFunction "param_1" "param_2"
```


Fonction - Retour de fonction

Problématique : Comment récupérer des informations retourner par une fonction ?

- Solution 1 :

```
1 res=""
2 MaFunction () {
3     res="salut"
4 }
5
6 MaFunction
7 echo $res
```

- Solution 2 :

```
1 MaFunction () {
2     echo "results"
3 }
4
5 res=`MaFunction`
6 echo $res
```

Problématique : Comment traiter les variables locales ?

```
1 MaFunction () {  
2     local res="salut"  
3 }  
4  
5 MaFunction  
6 echo $res
```

Enchaînements de commandes

Enchaînement simple : les commandes sont lancées les unes à la suite des autres (exécution successives)

```
1 #!/bin/bash
2
3 cmd_1
4 cmd_2
5 cmd_2
6
7 echo "fini"
```

Listing 13: "Enchaînement simple de plusieurs commandes"

Enchaînement simultanées : les programmes sont lancés en parallèle

```
1 #!/bin/bash
2
3 cmd_1 &
4 cmd_2 &
5 cmd_3 &
6
7 wait
8 echo "fini"
```

Listing 14: "Enchaînement simultanées de plusieurs commandes"

Enchaînements conditionnels - Partie 1

Enchaînement conditionnels et : Dans ce cas, la commande `comP` ne s'exécute que si `com(P-1)` s'est soldée par un succès.

```
1 #!/bin/bash
2
3 cmd_1 && cmd_2 && cmd_2
```

Enchaînement conditionnels alternative : les commandes `com1` jusqu'à `comN` sont exécutées successivement tant qu'aucune ne se termine correctement. Dès qu'une des commandes se solde par un succès, la commande alternative est terminée.

```
1 #!/bin/bash
2
3 cmd_1 || cmd_2 || cmd_3
```

Enchaînements conditionnels - Partie 2

Enchaînement conditionnels et

```
1 #!/bin/bash
2
3 echo "toto" && echo "tata" && echo "titi"
```

```
toto
tata
titi
```

Enchaînement conditionnels alternative

```
1 #!/bin/bash
2
3 echo "toto" || echo "tata" || echo "titi"
```

```
toto
```

Enchaînements conditionnels - Partie 3

Enchaînement conditionnels et

```
1 #!/bin/bash
2
3 note=8
4 [ $note -gt 10 ] && echo "vous avez la
    moyenne"
```

Enchaînement conditionnels alternative

```
1 #!/bin/bash
2
3 note=8
4 [ $note -gt 10 ] || echo "vous n'avez pas la
    moyenne"
```

vous n'avez pas la moyenne

Section 3

Les Tableaux en Bash : Arrays et Tableaux Associatifs

Introduction aux Tableaux en Bash

- Les tableaux permettent de stocker plusieurs valeurs dans une seule variable.
- Deux types de tableaux :
 - **Tableaux indexés** : indexés numériquement (0, 1, 2, ...).
 - **Tableaux associatifs** : indexés par des clés arbitraires (disponibles à partir de Bash 4).
- Utile pour organiser et manipuler des données dans vos scripts.

Tableaux Indexés : Déclaration et Accès

- Déclaration et initialisation d'un tableau indexé :

```
1 # Déclaration d'un tableau  
2 fruits=("pomme" "banane" "cerise")
```

- Accès aux éléments :

```
1 echo ${fruits[0]} # affiche "pomme"  
2 echo ${fruits[1]} # affiche "banane"
```

Itération sur un Tableau Indexé

- Pour parcourir tous les éléments d'un tableau :

```
1 for fruit in "${fruits[@]}"; do
2     echo "Fruit : $fruit"
3 done
```

Opérations sur les Tableaux Indexés

- Taille d'un tableau :

```
1 echo "Nombre de fruits : ${#fruits[@]}"
```

- Récupérer tous les index :

```
1 echo "Index du tableau : ${!fruits[@]}"
```

- Ajouter un élément :

```
1 fruits+=("orange")
```

Tableaux Associatifs : Introduction

- Disponibles à partir de Bash 4.
- Permettent d'utiliser des clés personnalisées plutôt que des indices numériques.

Tableaux Associatifs : Déclaration et Accès

- Déclaration d'un tableau associatif :

```
1 declare -A capitales
2 capitales["France"]="Paris"
3 capitales["Espagne"]="Madrid"
4 capitales["Italie"]="Rome"
```

- Accès aux éléments :

```
1 echo "La capitale de la France est ${capitales["France"]}"
```

Itération sur un Tableau Associatif

- Parcourir les clés et accéder aux valeurs correspondantes :

```
1 for pays in "${!capitales[@]}"; do
2     echo "La capitale de $pays est ${capitales[$pays]}"
3 done
```

Conclusion

- Les tableaux en Bash sont des outils puissants pour manipuler des collections de données.
- Les tableaux indexés sont simples et efficaces pour des listes ordonnées.
- Les tableaux associatifs offrent une flexibilité supplémentaire en permettant l'utilisation de clés arbitraires.
- N'hésitez pas à expérimenter ces fonctionnalités pour structurer et simplifier vos scripts.

Section 4

Les Variables d'Environnement en Bash

Introduction aux Variables d'Environnement

- Les variables d'environnement stockent des informations sur la configuration du système et de l'utilisateur.
- Elles influencent le comportement du shell et des programmes exécutés.
- Exemples courants : **PATH**, **PS1**, **HOME**, **USER**, etc.

La Variable PS1 (Prompt)

- **PS1** définit l'apparence de l'invite de commande.
- Exemple de valeur par défaut :

```
1 echo $PS1
2 # Exemple de résultat : \u@\h:\w$
3 # \u : nom d'utilisateur, \h : nom d'hôte, \w : répertoire courant
```

- Personnalisation de l'invite :

```
1 export PS1="\[\033[01;32m\]\u@\h:\w$ \[\033[00m\]"
```

La Variable PATH

- **PATH** indique les répertoires dans lesquels le shell recherche les exécutables.
- Pour afficher son contenu :

```
1 echo $PATH
```

- Pour ajouter un nouveau répertoire :

```
1 export PATH=$PATH:/chemin/vers/nouveau/dossier
```

Autres Variables Utiles

- **HOME** : Répertoire personnel de l'utilisateur.
- **USER** : Nom de l'utilisateur courant.
- **SHELL** : Chemin vers le shell utilisé.
- **PWD** : Répertoire de travail actuel.
- **LANG** : Paramètres régionaux (langue, encodage, etc.).

```
1 echo "Home : $HOME"  
2 echo "User : $USER"  
3 echo "Shell : $SHELL"  
4 echo "Répertoire courant : $PWD"
```

Variables Personnalisées et Exportation

- Vous pouvez créer vos propres variables dans le shell.
- Pour qu'une variable soit accessible aux programmes lancés par le shell, il faut l'exporter.

```
1 # Création d'une variable personnalisée
2 MON_VAR="Hello Bash"
3
4 # Affichage de la variable
5 echo $MON_VAR
6
7 # Exportation de la variable pour qu'elle soit disponible aux sous-processus
8 export MON_VAR
```

Astuce : Personnaliser Votre Environnement

- Modifiez le fichier `.bashrc` (ou `.bash_profile` selon votre distribution) pour personnaliser votre environnement de travail.
- Par exemple, ajoutez vos personnalisations de **PS1** et de **PATH** dans ce fichier.

```
1 # Exemple d'ajout dans ~/.bashrc
2 export PS1="\[\033[01;34m\]\u@\h:\w\$ \[\033[00m\]"
3 export PATH=$PATH:/chemin/vers/scripts
```

Conclusion

- Les variables d'environnement telles que **PS1** et **PATH** sont essentielles pour configurer et personnaliser votre shell.
- Vous pouvez définir et exporter vos propres variables pour améliorer votre expérience.
- N'hésitez pas à explorer et tester différentes configurations pour trouver celle qui vous convient le mieux.

Section 5

Compression des données

- **Définition :** La compression consiste à réduire l'espace occupé par un ensemble de données en éliminant les redondances et, éventuellement, en simplifiant certaines informations.
- **Objectifs :**
 - Optimiser le stockage.
 - Réduire le temps de transmission.
 - Préserver, selon les cas, la qualité ou la fidélité des données.
- **Types de compression :**
 - **Sans perte** (lossless) : La décompression restitue exactement les données originales (ex. fichiers textes, images PNG, archives ZIP).
 - **Avec perte** (lossy) : La décompression fournit une approximation des données originales, en éliminant des informations jugées peu perceptibles (ex. JPEG, MP3, MPEG).

Compression sans perte et ses Méthodes

- **Principe général :** Retrouver les données originales sans aucune modification après décompression.
- **Méthodes courantes :**
 - **Codage par répétition (Run-Length Encoding, RLE)**

Remplace les suites de caractères identiques par un couple (nombre d'occurrences ; symbole).
Exemple : "AAAAABBBCCDAA" devient "A5 B3 C2 D1 A2".
Avantage : Très efficace sur des données avec de longues séquences identiques.
Limite : Peut augmenter la taille si les répétitions sont rares.
 - **Codage entropique**

Utilise les statistiques sur la source pour attribuer des codes de longueur variable aux symboles (exemple : le codage de Huffman).
Principe : Les symboles fréquents reçoivent des codes courts pour se rapprocher de la limite de Shannon.
Variante avancée : Le codage arithmétique, plus précis mais plus complexe.
 - **Codage par dictionnaire**

Identifie des séquences récurrentes et les remplace par des références dans un dictionnaire (ex. LZ77, LZ78, LZW).
Exemple d'application : Le format GIF utilise LZW pour compresser les images.

Compression sous Linux : Tar, Gzip et Bzip2

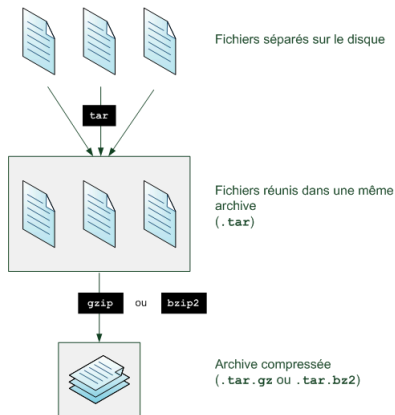


Illustration de l'outil tar

- **Tar :**

- Regroupe plusieurs fichiers et dossiers en une seule archive.
- Préserve la structure, les permissions et les métadonnées.

- **Compression de l'archive :**

- **Gzip** utilise l'algorithme DEFLATE (basé sur LZ77 et le codage de Huffman) pour compresser rapidement.
- **Bzip2** offre un taux de compression supérieur grâce à la transformation de Burrows-Wheeler suivie d'un codage entropique, au prix de performances moindres.

Créer une Archive avec Tar

Commande de base :

```
invite/> tar -cvf nom_archive.tar dossier_ou_fichier
```

Options principales :

- **-c** : Créer une archive.
- **-v** : Mode verbeux (affiche les opérations réalisées).
- **-f** : Spécifie le nom du fichier archive.

Remarque : Tar regroupe les fichiers sans les compresser. Pour compresser, on combinera tar avec gzip ou bzip2.

Archiver plusieurs fichiers ou dossiers :

```
invite/> tar -cvf archive.tar dossier1 dossier2 fichier1 fichier2
```

Afficher le contenu d'une archive sans extraction :

```
invite/> tar -tf archive.tar  
fichier1  
dossier1/fichier2  
...
```

Ajouter un fichier à une archive existante :

```
invite/> tar -rvf archive.tar fichier_ajouté
```

- **-r** : Ajoute de nouvelles entrées à l'archive existante.

Extraire une Archive

Commande d'extraction :

```
invite/> tar -xvf archive.tar
```

Options :

- **-x** : Extraire les fichiers de l'archive.
- **-v** : Affichage détaillé des opérations.
- **-f** : Spécifie le nom de l'archive.

Attention : L'extraction respecte la structure des dossiers et les permissions enregistrées.

Compresser et Décompresser une Archive

Compression de l'archive :

```
invite/> gzip archive.tar    ⇒  archive.tar.gz  
invite/> bzip2 archive.tar   ⇒  archive.tar.bz2
```

Décompression de l'archive :

```
invite/> gunzip archive.tar.gz  
invite/> bunzip2 archive.tar.bz2
```

Note : Gzip privilégie la rapidité, tandis que bzip2 vise un taux de compression plus élevé.

Archiver et Compresser en une seule Commande

Avec gzip :

```
invite/> tar -zcvf archive.tar.gz dossier/
```

- **-z** : Indique à tar d'utiliser gzip pour compresser l'archive.

Pour décompresser :

```
invite/> tar -zxvf archive.tar.gz
```

Avec bzip2 : Remplacez l'option **-z** par **-j** et l'extension en **.tar.bz2**.